

PROJECT PROPOSAL

RideBoard

A Ride-Sharing Web Application

Instructor: Sir Talha Shahid

Course: Database Systems

Team: Huzaifa Ahmed Bari (24k-0847)
Shoaib Hayat (24k-1028)

Date: 10th May, 2026

1. Executive Summary

RideBoard is a full-stack, scheduled carpooling web application. The platform addresses the daily commuting challenge faced by students and professionals by connecting drivers who have vacant seats with riders travelling along the same route. Unlike on-demand ride-hailing services, RideBoard focuses on pre-planned, scheduled rides—enabling users to post and book journeys in advance, reducing transport costs, easing traffic, and building community trust through a transparent review system.

The system is built on a modern technology stack: a React.js frontend, a Node.js/Express.js backend API, and Supabase (PostgreSQL) for data persistence, authentication, and real-time messaging. Geospatial capabilities are provided by OSRM for route polyline generation and Nominatim for geocoding, with OpenStreetMap and Leaflet for mapping. The platform features automated ride lifecycle management via Supabase’s `pg_cron` extension, ensuring rides are automatically marked expired once their departure time passes.

2. Problem Statement

Urban commuting in Pakistan—particularly in cities such as Karachi—is characterised by heavy traffic, high fuel costs, and a lack of affordable, reliable transportation options. A significant proportion of daily commuters (students, office workers, and professionals) travel along similar routes at similar times, yet each individual travels alone in their own vehicle or relies on expensive ride-hailing services.

The core problems this project addresses are:

- High individual transport costs due to single-occupancy vehicle use.
- Absence of a trusted, community-oriented carpooling platform tailored to scheduled routes.
- Lack of real-time communication tools between carpool participants.
- No mechanism for building user trust and accountability in informal ride-sharing arrangements.
- Traffic congestion worsened by low vehicle occupancy rates.

Existing solutions such as Careem and Uber are commercial ride-hailing services—they charge high per-kilometre fares and do not support community-based seat-sharing among peers. RideBoard fills this gap by providing a peer-to-peer, scheduled carpooling experience with built-in trust features.

3. Proposed Solution

RideBoard is a web application that allows users to register as both drivers and riders. Drivers post scheduled rides by selecting their origin and destination on an interactive map, setting a departure time, specifying the fare per seat, and defining the vehicle's capacity. Riders can then search for available rides by proximity and departure time, book a seat, and communicate with the driver and fellow riders through a dedicated group chat.

3.1 Core Features

- **User Registration & Authentication:** Secure sign-up and login via Supabase Auth (email/password).
- **Ride Posting:** Map-based origin/destination selection with OSRM route visualisation, fare and seat configuration.
- **Ride Search:** Proximity-based search by location and time for active, non-full rides.
- **Real-Time Booking:** Instant seat reservation with live seat count updates via Supabase real-time.
- **Group Chat:** Per-ride private group chat for the driver and all confirmed riders; cancelled riders are shown as 'Cancelled Rider' to preserve chat history.
- **Trust & Reviews:** Post-ride rating and review system for building community accountability.
- **Automated Ride Expiry:** `pg_cron` job runs every minute to expire rides past their departure time.

3.2 Technical Architecture

The application follows a three-tier architecture:

- **Presentation Layer:** React.js with Vanilla CSS. Leaflet.js is used for interactive map rendering.
- **Application Layer:** Node.js/Express.js acts as a thin REST API, handling business logic and proxying requests to Supabase.
- **Data Layer:** Supabase (hosted PostgreSQL) manages persistence, Row Level Security, authentication, and real-time WebSocket subscriptions.

3.3 Technology Stack

Layer	Technology
Frontend Framework	React.js
Frontend Styling	Vanilla CSS
Map Rendering	Leaflet.js
Backend Runtime	Node.js
Backend Framework	Express.js
Database & Auth	Supabase (PostgreSQL)
Route Calculation	OSRM
Geocoding	Nominatim
Scheduling	pg_cron
Version Control	Git / GitHub

Table 1: Technology Stack

4. Database Design

The RideBoard database consists of five primary tables designed to support the full ride lifecycle from posting through booking, messaging, and review.

4.1 Tables Summary

Table	Primary Key	Description
Users	id (UUID)	Registered user accounts—both drivers and riders
Rides	id (UUID)	Ride listings posted by drivers
bookings	id (UUID)	Seat reservations linking riders to rides
messages	id (UUID)	Group chat messages scoped to a ride
reviews	id (UUID)	Post-ride ratings and feedback between participants

Table 2: Database Tables Summary

4.2 Users Table

Column	Data Type	Constraints	Description
id	UUID	PK, NOT NULL	Linked to Supabase Auth UID
name	TEXT	NOT NULL	Full name of the user
email	TEXT	UNIQUE, NOT NULL	Email address
phone	TEXT	–	Optional contact number
created_at	TIMESTAMP	DEFAULT NOW()	Account creation timestamp

Table 3: Users Table

4.3 Rides Table

Column	Data Type	Constraints	Description
id	UUID	PK, NOT NULL	Auto-generated primary key
poster_id	UUID	FK $\rightarrow$ users.id	The driver who posted the ride
origin_address	TEXT	NOT NULL	Human-readable origin location
destination_address	TEXT	NOT NULL	Human-readable destination
origin_lat	FLOAT	NOT NULL	Origin latitude coordinate
origin_lng	FLOAT	NOT NULL	Origin longitude coordinate
destination_lat	FLOAT	NOT NULL	Destination latitude coordinate
destination_lng	FLOAT	NOT NULL	Destination longitude coordinate
start_time	TIMESTAMP	NOT NULL	Scheduled departure date and time
fare	NUMERIC(8,2)	NOT NULL, >0	Fare per seat in PKR
total_seats	INTEGER	NOT NULL, 1–8	Maximum passenger capacity
seats_remaining	INTEGER	NOT NULL, ≥ 0	Current available seats
status	TEXT	DEFAULT 'active'	active full expired
created_at	TIMESTAMP	DEFAULT NOW()	Record creation timestamp

Table 4: Rides Table

4.4 Bookings Table

Column	Data Type	Constraints	Description
id	UUID	PK, NOT NULL	Auto-generated primary key
ride_id	UUID	FK → rides.id	The ride being booked
rider_id	UUID	FK → users.id	The rider making the booking
status	TEXT	DEFAULT 'confirmed'	confirmed cancelled
created_at	TIMESTAMP	DEFAULT NOW()	Booking creation timestamp

Table 5: Bookings Table

4.5 Messages Table

Column	Data Type	Constraints	Description
id	UUID	PK, NOT NULL	Auto-generated primary key
ride_id	UUID	FK → rides.id	Scopes message to a ride's chat
sender_id	UUID	FK → users.id	The user who sent the message
content	TEXT	NOT NULL	Message body text
created_at	TIMESTAMP	DEFAULT NOW()	Message send timestamp

Table 6: Messages Table

4.6 Reviews Table

Column	Data Type	Constraints	Description
id	UUID	PK, NOT NULL	Auto-generated primary key
ride_id	UUID	FK → rides.id	The ride context for the review
reviewer_id	UUID	FK → users.id	User submitting the review
reviewee_id	UUID	FK → users.id	User being reviewed
rating	INTEGER	CHECK (1–5)	Star rating from 1 to 5
comment	TEXT	–	Optional written feedback
created_at	TIMESTAMP	DEFAULT NOW()	Review submission timestamp

Table 7: Reviews Table

5. Constraints and Data Integrity

Table	Constraint Type	Rule
rides	CHECK	total_seats BETWEEN 1 AND 8
rides	CHECK	seats_remaining $\geq$ 0
rides	CHECK	seats_remaining $\leq$ total_seats
rides	CHECK	fare > 0
rides	CHECK	status IN ('active', 'full', 'expired')
rides	CHECK	start_time > created_at
bookings	CHECK	status IN ('confirmed', 'cancelled')
bookings	UNIQUE	(ride_id, rider_id) – one booking per rider per ride
reviews	CHECK	rating BETWEEN 1 AND 5
reviews	CHECK	reviewer_id $\neq$ reviewee_id
reviews	UNIQUE	(ride_id, reviewer_id, reviewee_id)
All	FK	ON DELETE CASCADE for ride-scoped data

Table 8: Constraints and Data Integrity Rules

6. Scheduled Automation – pg_cron

The `pg_cron` extension (enabled in Supabase) is used to register a scheduled database job that runs every minute. This job expires all rides whose departure time has passed and whose status is still `active` or `full`:

Schedule: Every minute

SQL executed:

```
UPDATE rides
SET status = 'expired'
WHERE start_time < NOW()
  AND status IN ('active', 'full');
```

7. Project Scope

7.1 In Scope

- User registration, authentication, and profile management.
- Driver: post rides with map-based origin/destination, departure time, fare, and seat capacity.
- Rider: search rides by proximity and time, view route on map, book seats.
- Real-time seat availability updates.
- Per-ride private group chat for driver and confirmed riders.
- User rating and review system post-ride.
- Automated ride expiry via `pg_cron`.

7.2 Out of Scope

- Real-time vehicle GPS tracking during an active ride.
- In-app payment processing or fare splitting.
- Native iOS or Android mobile applications.
- Admin/moderation dashboard.
- Ride matching based on dynamic traffic or AI algorithms.

8. Conclusion

RideBoard addresses a clear and prevalent problem in urban commuting by offering a trusted, scheduled carpooling solution built on proven modern web technologies. RideBoard demonstrates how a scheduled carpooling model can be implemented cleanly with modern open-source tools and free-tier cloud services. The architecture is significantly simpler than conventional ride-hailing systems while remaining practical and scalable. The project provides hands-on experience with spatial database queries, REST API design, third-party service integration, and interactive map development.

The platform’s emphasis on community trust—through verified accounts, real-time communication, and a post-ride review system—distinguishes it from commercial ride-hailing alternatives.